

Algorithm Design and Analysis - Homework 12

Ewan Pedersen

December 5, 2025

Abstract

Show the steps of deriving your answers. Points will be deducted for answers without adequate steps discussed. Submit your homework via Brightspace as one PDF or Word document.

1 Self Reducibility

Show how the optimization version of vertex cover problem is polynomial time-reducible to its decision version. The optimization version, known as min vertex cover problem, is “given a graph G , find the minimum set of vertices that cover all the edges in G ”. The decision version is “given a graph G and a constant k , is there a vertex cover of at most k vertices?”. Here, we say a vertex “covers” an edge if and only if the vertex is an end-node of the edge.

1.1 Reduction to Decision Version

We start with the two phases, finding the minimum size using binary search, and constructing an actual minimum vertex using the vertex elimination.

Using the given graph $G = (V, E)$ with value of $n = |V|$ vertices, we use binary search with the following decision oracle:

Initialize: $\ell = 0, r = n$ (1)

While $\ell < r$: (2)

$m = \lfloor (\ell + r)/2 \rfloor$ (3)

if DEC-VC(G, m) = YES : (4)

$r = m$ (5)

else : (6)

$\ell = m + 1$ (7)

Return $k^* = \ell$ (8)

This yields a big O complexity of $O(\log n)$ calls to DEC-VC, where each takes polynomial time.

Now that we know the value of k^* , we can construct an actual minimum vertex cover C :

$$C = \emptyset \tag{9}$$

$$k = k^* \tag{10}$$

$$V' = V \text{ (remaining vertices)} \tag{11}$$

$$\text{For each vertex } v \in V : \tag{12}$$

$$G' = G[V' \setminus \{v\}] \text{ (induced subgraph without } v) \tag{13}$$

$$\text{if DEC-VC}(G', k - 1) = \text{NO} : \tag{14}$$

$$C = C \cup \{v\} \tag{15}$$

$$k = k - 1 \tag{16}$$

$$V' = V' \setminus \{v\} \tag{17}$$

$$\text{Return } C \tag{18}$$

We notice that when $\text{DEC-VC}(G', k - 1)$ returns NO, then vertex v MUST be in every vertex cover of the size k for current graph.

Now we have the information necessary for a proof via contradiction:

We suppose that v is not needed. If this is the case, then there exists a vertex cover of size k not containing v , where this cover only uses vertices from $V' \setminus \{v\}$. Therefore, G' has a vertex cover of size $\leq k - 1$, which is a **contradiction**.

This algorithm terminates with $|C| = k^*$ because we only add a vertex to C when its essential, we decrement k each time we add a vertex, and we started with exactly k^* slots to fill.

This yields the final complexity:

$$\text{Phase 1: } O(\log n) \text{ oracle calls} \tag{19}$$

$$\text{Phase 2: } O(n) \text{ oracle calls (one per vertex)} \tag{20}$$

$$\text{Total: } O(n + \log n) = O(n) \text{ oracle calls} \tag{21}$$

Given that each phase call is completed in polynomial time, that means the full reduction also runs in polynomial time. ■

2 Set Packing and Independent Set

Prove that $\text{SET-PACKING} \leq_p \text{INDEPENDENT-SET}$, that is, the SET-PACKING decision problem is polynomial-time reducible to the INDEPENDENT-SET decision problem. Feel free to use an illustration if it helps. The definitions of these two decision problems are summarized below.

- SET-PACKING: Given a set U of elements, a set of subsets $S_1, S_2, \dots, S_m$ of U , and an integer k , does there exist a set of at least k subsets that are pairwise disjoint (i.e., intersection = $\emptyset$ between every pair)?

- **INDEPENDENT-SET**: Given a graph $G = (V, E)$ and an integer k , is there a subset of vertices $S \subseteq V$ such that $|S| \geq k$ and, for each edge in E , at most one – but not both – of its end nodes is in S ?

(Note we've already proved in class that $\text{INDEPENDENT-SET} \leq_p \text{SET-PACKING}$, so this proof leads to it that $\text{INDEPENDENT-SET} \equiv_p \text{SET-PACKING}$, that is, these two problems are equally hard computationally.)

Make sure to give a complete proof based on the Karp reduction definition (which is for decision problems) by including the two parts—(1) the polynomial-time construction algorithm and (2) the equivalence of solutions to the two problems.

2.1 Constructing the Reduction

Given a **SET-PACKING** instance $(U, \{S_1, S_2, \dots, S_m\}, k)$, construct an **INDEPENDENT-SET** instance (G, k) as follows:

Graph $G = (V, E)$ where: (22)

$$V = \{v_1, v_2, \dots, v_m\} \quad (\text{one vertex per subset}) \quad (23)$$

$$E = \{(v_i, v_j) : S_i \cap S_j \neq \emptyset, i \neq j\} \quad (\text{edge iff subsets intersect}) \quad (24)$$

Keeping the same value of k . In essence, the two subsets are disjoint, their corresponding vertices are not connected by an edge.

A demo would look something like:

$$U = \{1, 2, 3, 4, 5\}$$

$$S_1 = \{1, 2\}$$

$$S_2 = \{2, 3\}$$

$$S_3 = \{3, 4\}$$

$$S_4 = \{4, 5\}$$

$$S_5 = \{1, 5\}$$

$$k = 2$$

Their intersections would look like:

$$S_1 \cap S_2 = \{2\} \neq \emptyset \rightarrow \text{edge}(v_1, v_2)$$

$$S_2 \cap S_3 = \{3\} \neq \emptyset \rightarrow \text{edge}(v_2, v_3)$$

$$S_3 \cap S_4 = \{4\} \neq \emptyset \rightarrow \text{edge}(v_3, v_4)$$

$$S_4 \cap S_5 = \{5\} \neq \emptyset \rightarrow \text{edge}(v_4, v_5)$$

$$S_1 \cap S_5 = \{1\} \neq \emptyset \rightarrow \text{edge}(v_1, v_5)$$

All other pairs are disjoint. See Figure 1 for a rough visualization of this graph.

To prove the correctness, we must now define our theorem that SET-PACKING has a solution of size $\geq k \iff G$ has an independent set of size $\geq k$.

For the forward direction, we suppose that $\{S_{i_1}, S_{i_2}, \dots, S_{i_k}\}$ is a set packing:

$$\text{For all } p \neq q : S_{i_p} \cap S_{i_q} = \emptyset \quad (25)$$

$$\implies \text{No edge } (v_{i_p}, v_{i_q}) \in E \quad (26)$$

$$\implies \{v_{i_1}, v_{i_2}, \dots, v_{i_k}\} \text{ is an independent set in } G \quad (27)$$

For the backward direction, we suppose that $\{v_{j_1}, v_{j_2}, \dots, v_{j_k}\}$ is an independent set of G .

$$\text{For all } p \neq q : (v_{j_p}, v_{j_q}) \notin E \quad (28)$$

$$\implies S_{j_p} \cap S_{j_q} = \emptyset \quad (\text{by construction}) \quad (29)$$

$$\implies \{S_{j_1}, S_{j_2}, \dots, S_{j_k}\} \text{ is a set packing} \quad (30)$$

Now to analyze its complexity:

$$\text{Create } m \text{ vertices: } O(m) \quad (31)$$

$$\text{Check all pairs for intersection: } O(m^2 \cdot |U|) \quad (32)$$

$$\text{Total: } O(m^2 \cdot |U|) = \text{polynomial in input size} \quad (33)$$

Given that this reduction is computable in polynomial time (and also preserves solutions), we know:

$$\text{SET-PACKING} \leq_p \text{INDEPENDENT-SET}$$

Combining with the result from class of INDEPENDENT-SET $\leq_p$ SET-PACKING, we now know:

$$\text{SET-PACKING} \equiv_p \text{INDEPENDENT-SET}$$

And thus the problems are computationally equivalent ■.

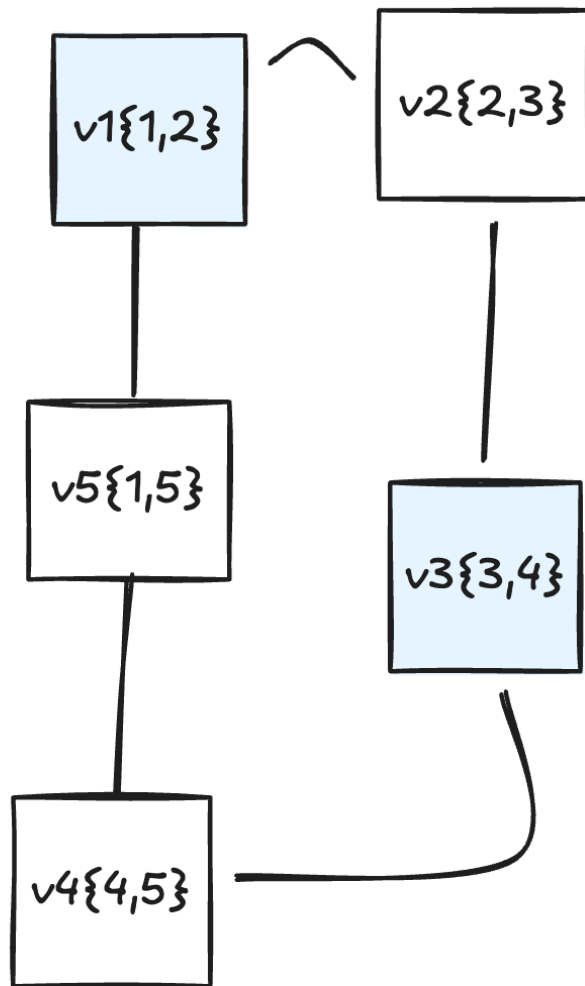


Figure 1: Resultant Graph (rough)